SSH : mise en place et sécurisation d'OpenSSH (TP)

Nom Prénom : **Dylan Spinelli**

BTS SIO SISR

Paris Ynov Campus

Sommaire

A- Introduction..... 1

B- Mise en place..... 2

1- Mettre en place deux machines Debian 11 nom de code bullseye..... 2

2- Mettre à jour les deux machines..... 2

3- Changer le nom des deux machines pour sshclient et sshserver..... 2

4- Installer le paquet :..... 2

Vérification..... 2

5- Créez un utilisateur userserver sur le serveur sshserver..... 3

6- Les fichiers de configuration..... 3

7- Connexion au :..... 5

C- Création d'une identité NUMERIQUE pour les comptes utilisateurs ssh en créant une paire de clés asymétrique..... 9

1- Création d'une identité NUMERIQUE pour userclient en créant une paire de clés asymétrique sur sshclient..... 9

2- J'envoie la clé publique à userserver sur sshserver..... 11

D- Passphrase et agent SSH..... 12

a- Création d'une nouvelle instance bash..... 12

b- Charger la clé dans la mémoire de l'agent..... 12

c- Connexion ssh en utilisant l'agent ssh avec la Passphrase en mémoire..... 13

d- Autres options..... 13

E- Créations du fichier config..... 13

a- Créer le fichier config dans .ssh..... 13

b- Modifier le fichier config..... 13

F- La double Authentification google..... 14

G- Création d'une bannière..... 17

H- Connexion ssh avec une solution mobile..... 17

a- Connexion avec une redirection de port..... 17

b- Connexion avec un partage de connexion 4G..... 19

I- Les Ports de communication:..... 21

A- Introduction

(SSH) Secure Shell est un protocole de communication Sécurisé. Ce protocole de connexion exige un échange de clés de chiffrement pour toutes connexions, tous les segments TCP sont **authentifiés et chiffrés**. Toutes attaque de types ***man-in-the-middle*** (**MITM**)

Le protocole SSH a été conçu pour remplacer les différents protocoles non chiffrés comme rlogin, telnet, rcp et rsh.

OpenSSH utilise la **cryptographie*** asymétrique comme mécanisme d'authentification. OpenSSH gère les clés RSA, les clés DSA et les clés DSA basées sur les courbes elliptiques. OpenSSH reconnaît aussi les certificats X509 et les fichiers au format PKCS#12.

*La cryptographie sert principalement 3 objectifs :

a- La confidentialité
b- L'authentification (identification des partenaires, contrôle d'accès aux ressources et données, non répudiation de l'origine de l'information)
c- L'intégrité des données (données non altérées accidentellement ou frauduleusement lors du transfert ou sur place)

OpenSSh est, on l'a dit, une alternative à SSH. Il est également tout indiqué pour remplacer

La principale différence entre OpenSSH et SSH est bien entendu le fait qu'OpenSSH soit libre. Il y a aussi quelques petites différences de configuration.

Modèle client/serveur

a- Comme tout protocole qui se respecte, OpenSSH est composé d'un serveur et d'un client :
b- Le serveur (
c- écoute sur un port (22) par default. Pour une connexion entrante, il vérifie que c'est bien un client ssh qui est à l'autre bout, demande à l'utilisateur de s'identifier via paire de clés asymétriques ou un mot de passe. Dans le premier cas, le serveur va lire un fichier de clés publiques (~/.ssh/authorized_keys) dans le répertoire home de l'user annoncé. S'il y trouve une clé ayant le même nom que celle proposée par le client, il va les vérifier par l'envoi d'un 'challenge' crypté avec la clé publique. Seule la clé privée pourra le décrypter, et elle renverra le challenge décrypté, prouvant que le client connait la clé privée, mais sans jamais la dévoiler.
   Dans le deuxième cas, le serveur vérifie que le mot de passe et le nom correspondent à un utilisateur de la machine. Cette méthode n'est utilisée que si la méthode par paire de clés a échoué. Le mot de passe étant lui-même crypté durant le transfert, il n'est pas possible de l'intercepter en clair. Si le client est autorisé, la machine locale est disponible en interactif, avec les droits de la personne entrée.
d- Le client sert à se connecter au serveur. C'est lui qui choisit les algorithmes (et par là, la version du protocole). Si le serveur ne reconnait pas l'algorithme demandé, c'est au client d'en proposer un autre.
e- Certaines options peuvent être configurées des deux côtés, serveur et client. La règle qui prévaut alors est celle du 'Tout ce qui n'est pas interdit est autorisé'. Par exemple, le client autorisera les applications X-Windows à passer par le tunnel ssh, que si la configuration du serveur ne l'interdit pas expressément.

B- Mise en place

1- Mettre en place deux machines Debian 11 nom de code bullseye

Dans l'environnement Sitka (plateforme 1), les machines du TP sont placées dans le VLAN 20 - Equipes (172.20.2.0/24) avec des adresses fixes situées en dehors de l'étendue DHCP (172.20.2.2 → 172.20.2.10) : sshserver 172.20.2.30, sshclient 172.20.2.31 et w2019 172.20.2.32. La passerelle est le routeur virtuel HSRP 172.20.2.254 et le DNS est hermes (172.20.1.2).protocole de communicationclés de chiffrementsegmentsTCP

Cloner deux machines à partir de la machine d'origine

2- Mettre à jour les deux machines

```
root💀debian:~# apt update
root💀debian:~# apt upgrade
```

meme

3- Changer le nom des deux machines pour sshclient et sshserver

```
root💀debian:~# hostnamectl set-hostname sshclient
```

NAAPT

```
root💀debian:~# hostnamectl set-hostname sshserver
```

On vérifiera que les noms ont changé dans le fichier hostname

```
root💀debian:~# vim /etc/hostname
```

Ou avec la commande *hostnamectl*

```
root💀sshserver:~# hostnamectl
   Static hostname: sshserver
         Icon name: computer-vm
```

4- Installer le paquet :

- Openssh-client sur le serveur sshclient
- Openssh-server sur le serveur sshserver

a- Sur la machine sshserver on installe le paquet openssh-server

```
root💀sshserver:~# apt install openssh-server -y
```

Vérification : on constate que le paquet openssh est installé

```
root💀sshcient:~# dpkg -l openssh*
Souhait=inconnU/Installé/suppRimé/Purgé/H=à garder
| État=Non/Installé/fichier-Config/dépaqUeté/échec-conFig/H=semi-installé/W=attend-traitement-déclenchements
|/ Err?=(aucune)/besoin Réinstallation (État,Err: majuscule=mauvais)
||/ Nom                Version      Architecture Description
+++-==================-============-============-=================================================================>
ii  openssh-client     1:8.4p1-5    amd64        secure shell (SSH) client, for secure access to remote machines
un  openssh-server     <aucune>     <aucune>     (aucune description n'est disponible)
ii  openssh-sftp-server 1:8.4p1-5   amd64        secure shell (SSH) sftp server module, for SFTP access from remote ma>
un  openssh-sk-helper  <aucune>     <aucune>     (aucune description n'est disponible)
lines 1-9/9 (END)
```

b- Sur la machine sshclient on installe le paquet openssh-client

```
root💀sshcient:~# apt install openssh-client -y
```

Vérification : on constate que le paquet openssh est install

Vérification

```
root💀sshclient:~# ssh -V
OpenSSH_8.4p1 Debian-5, OpenSSL 1.1.1k  25 Mar 2021
root💀sshclient:~# dpkg -l libssl*
Souhait=inconnU/Installé/suppRimé/Purgé/H=à garder
| État=Non/Installé/fichier-Config/dépaqUeté/échec-conFig/H=semi-installé/W=atten>
|/ Err?=(aucune)/besoin Réinstallation (État,Err: majuscule=mauvais)
||/ Nom            Version          Architecture Description
+++-==============-================-============-=============================>
ii  libssl1.1:amd64 1.1.1k-1+deb11u1 amd64        Secure Sockets Layer toolkit - >
lines 1-6/6 (END)
```

Les différentes commandes liées à ssh

```
root💀sshserver2:/usr/bin# ls *ssh*
ssh  ssh-add  ssh-agent  ssh-argv0  ssh-copy-id  ssh-keygen  ssh-keyscan
```

/cler

```
root💀sshserver:~# dpkg -s openssh-server
```

```
root💀sshserver:~# apt show openssh-server
```

On vérifie si le service ssh est démarré

```
root💀sshserver2:~# /etc/init.d/ssh status
```

Ou

```
root💀sshserver2:~# systemctl status ssh
```

ou

```
root💀sshserver:~# service ssh status
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enab>
     Active: active (running) since Sat 2021-09-25 21:15:40 CEST; 18s ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 2603 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2604 (sshd)
      Tasks: 1 (limit: 2303)
     Memory: 1.1M
        CPU: 24ms
     CGroup: /system.slice/ssh.service
             └─2604 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups

sept. 25 21:15:40 sshserver systemd[1]: Starting OpenBSD Secure Shell server...
sept. 25 21:15:40 sshserver sshd[2604]: Server listening on 0.0.0.0 port 22.
sept. 25 21:15:40 sshserver sshd[2604]: Server listening on :: port 22.
sept. 25 21:15:40 sshserver systemd[1]: Started OpenBSD Secure Shell server.
```

dp

a

5- Créez un utilisateur **userserver** sur le serveur **sshserver**
Et **userclient** sur le serveur **sshclient**

6- Les fichiers de configuration

a- Sur le serveur ssh

clea

```
root💀sshserver:~# ls -l /etc/ssh
total 608
-rw-r--r-- 1 root root 577771 13 mars   2021 moduli
-rw-r--r-- 1 root root   1650 13 mars   2021 ssh_config
drwxr-xr-x 2 root root   4096 13 mars   2021 ssh_config.d
-rw-r--r-- 1 root root   3289 13 mars   2021 sshd_config
drwxr-xr-x 2 root root   4096 13 mars   2021 sshd_config.d
-rw------- 1 root root    505 25 sept. 21:15 ssh_host_ecdsa_key
-rw-r--r-- 1 root root    176 25 sept. 21:15 ssh_host_ecdsa_key.pub
-rw------- 1 root root    411 25 sept. 21:15 ssh_host_ed25519_key
-rw-r--r-- 1 root root     96 25 sept. 21:15 ssh_host_ed25519_key.pub
-rw------- 1 root root   2602 25 sept. 21:15 ssh_host_rsa_key
-rw-r--r-- 1 root root    568 25 sept. 21:15 ssh_host_rsa_key.pub
```

Moduli : Il n'est pas fait pour être utiliser directement par l'utilisateur c'est un fichier qui est géré par ssh on peut le générer avec la commande ssh-keygen c'est un fichier qui va contenir des données utiles quand on va faire un échange avec la clé diffie-hellman

La clé diffie-hellman permet à deux correspondant de concevoir une clé symétrique en échangeant une quantité de donnée qui ne permet pas de calculer cette clé
Ssh_config : la configuration du client permet d'identifier les options configurées pour ssh

Sshd_config :

```
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes
```

Paire de clé qui identifiée
Rsa basé sur la factoris ation des lslsadd premiers
Ecdsa et ed25519 basé sur les courbes elliptiques

b- Sur le serveur client

On remarque qu'il existe juste un fichier et un répertoire
lsls

```
root💀sshclient:~# ls /etc/ssh
ssh_config  ssh_config.d
root💀sshclient:~#
```

7- Connexion au :

a- Serveur ssh

On se connecte toujours avec les comptes hébergés sur le serveur ssh et non les comptes locaux.
Depuis le client sshclient on va établir une connexion ssh au serveur ssh ; il existe deux façons,

#ssh -l <nom d'utilisateur sur le serveur ssh> <adresse ip du serveur ssh>

```
root💀sshclient:/etc/ssh# ssh -l userserver 192.168.44.152
The authenticity of host '192.168.44.152 (192.168.44.152)' can't be established.
ECDSA key fingerprint is SHA256:izZkTe4Q0Low872yX0Zf8e0S0WO5behllq9fIj1S11g.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
```

Ou

#ssh nom_utilisateur@adresse ip

Exemple

ssh
>>
cd

```
root💀sshclient:/etc/ssh# ssh userserver@192.168.44.152
The authenticity of host '192.168.44.152 (192.168.44.152)' can't be established.
ECDSA key fingerprint is SHA256:izZkTe4Q0Low872yX0Zf8e0S0WO5behllq9fIj1S11g.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Le serveur ssh envoi sa clé publique ainsi que l'empreinte de cette clé
On peut vérifier cette empreinte sur le serveur ssh avant d'accepter cette clé publique, après comparaison on remarque que les deux empreintes correspondent

sss

ou
s

```
root💀sshclient:~/.ssh# cat known_hosts
|1|/dkwRRR3fLLuNpbFHd3AgqOkiPc=|h0ifiUKMM0sHr5/fskD66sVJtAY= ecdsa-sha2-nistp256 AA
AAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBmDcokbIMrWStT4pMdxfzjJet+xkindg
MUbfqlVEO78ZdAZGDNF7d+YqzLoEGWqTE231QRmTRC/4VJUCuQNYzI=
root💀sshclient:~/.ssh#
```

ls

Pour effacer une clé

```
root💀debian:~#  ssh-keygen -f "/root/.ssh/known_hosts" -R "192.168.44.135"
```

On remarque que le prompt a changé on est bien sur le serveur ssh **(sshserver)**

Il faut noter qu'on ne peut pas se connecter par défaut en ssh avec le compte root ; pour des raisons de sécurité le compte est désactivé dans le fichier de configuration **sshd_config**

```
#LoginGraceTime 2m
#PermitRootLogin prohibit-password
```

```
root@sshclient:/etc/ssh# ssh root@192.168.44.152
The authenticity of host '192.168.44.152 (192.168.44.152)' can't be established.
ECDSA key fingerprint is SHA256:izZkTe4Q0Low872yX0Zf8e0S0WO5behllq9fIj1S11g.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.44.152' (ECDSA) to the list of known hosts.
root@192.168.44.152's password:
Permission denied, please try again.
```

B- Serveur client

Depuis le serveur ssh on va essayer de se connecter sur le client ssh, la connexion est refusée sur un client ssh on ne peut pas se connecter si le paquet openssh-server n'est pas installé.

```
root@sshserver:~# ssh userclient@192.168.44.158
ssh: connect to host 192.168.44.158 port 22: Connection refused
```

c- Serveur Windows 2019, 2022 et Windows 10

Installer OpenSSH avec les Paramètres Windows

Les deux composants OpenSSH client et serveur doivent être installés doivent être installés. Pour installer les composants OpenSSH :

1. Ouvrez **Paramètres**, sélectionnez **Applications > Applications et fonctionnalités**, puis **Fonctionnalités facultatives**.
2. Parcourez la liste pour voir si OpenSSH est déjà installé. Si ce n'est pas le cas, sélectionnez **Ajouter une fonctionnalité** en haut de la page, puis :
   - Recherchez **OpenSSH Client** et cliquez sur **Installer**.
   - Recherchez **OpenSSH Server** et cliquez sur **Installer**.

Une fois l'installation terminée, revenez à **Applications > Applications et fonctionnalités** et **Fonctionnalités facultatives**. Vous devriez voir OpenSSH dans la liste.



On peut installer ssh en ligne de commande sur un power shell

Installation d'OpenSSH en ligne de commande

Ouvrez le PowerShell en tant qu'administrateur. Pour s'assurer qu'OpenSSH est disponible, exécutez l'applet de commande suivante :Get-WindowsCapability -Online | ? Name -like 'OpenSSH*'

La sortie suivante doit être retournée si aucun n'est déjà installé :

```
Name : OpenSSH.Client~~~~0.0.1.0
State : NotPresent

Name  : OpenSSH.Server~~~~0.0.1.0
State : NotPresent
```

Installation des composants serveur ou client :

```
# Install the OpenSSH Client
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0

# Install the OpenSSH Server
Add-WindowsCapability -Online -Name OpenSSH.Server~~~~0.0.1.0
```

Quel que soit le composant, la sortie suivante devrait être retournée :

Démarrer et configurer OpenSSH Server

Pour la première utilisation de ssh, ouvrez PowerShell en tant qu'administrateur, il faut exécuter les commandes suivantes pour démarre

```
# démarrer le service sshd
Start-Service sshd

# configurer un demarrage automatique du service sshd:
Set-Service -Name sshd -StartupType 'Automatic'
```

Il faut désactiver le firewall ou il faut le configurer, pour le configurer il faut suivre les étapes suivantes

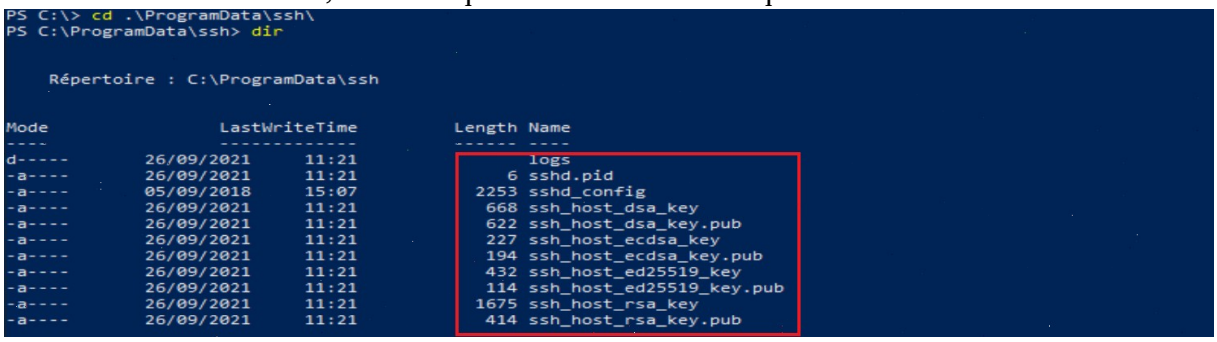
```
# autorisez ssh sur le firewall.

Get-NetFirewallRule -Name *ssh*

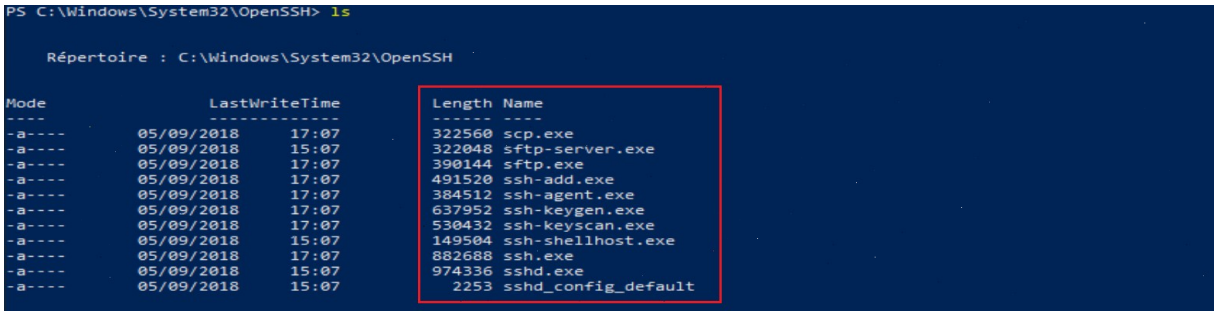
# la règle suivante doit être activée : "OpenSSH-Server-In-TCP", si cette règle n'existe pas il faut la créer

New-NetFirewallRule -Name sshd -DisplayName 'OpenSSH Server (sshd)' -Enabled True -Direction Inbound -Protocol TCP -Action Allow -LocalPort 22
```

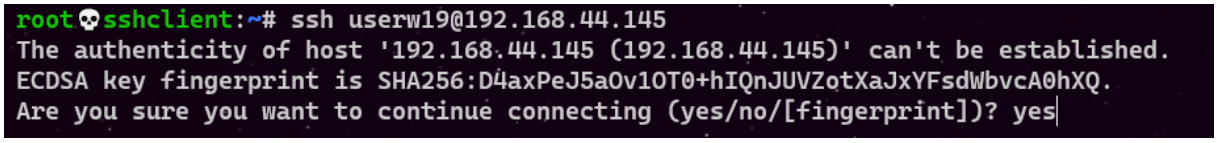
En vérifiant l'installation, on remarque les mêmes fichiers que sur linux



Dans system32 le dossier openssh regroupe toutes les commandes ssh



Maintenant on va tenter une connexion sur le serveur ssh Windows 2019 depuis le serveur sshclient
Avant il faut créer un utilisateur **userw19** avec **Azerty123** comme New-LocalUser -Name 'User02'

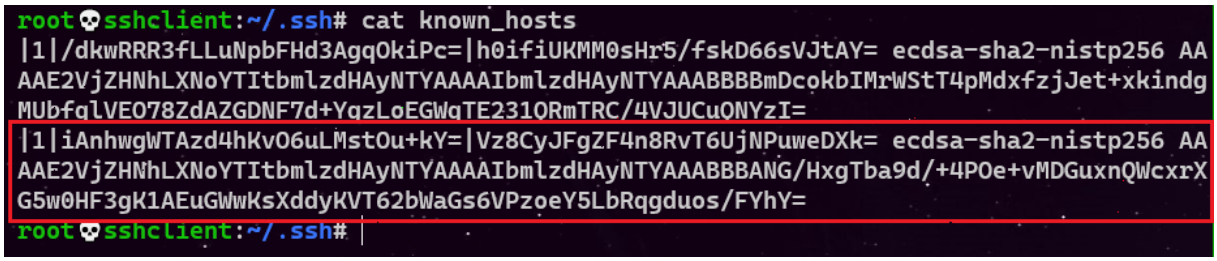


On tombe sur le prompt de notre machine Windows sur une console dos.
On peut lancer une console PowerShell à partir de cet endroit :

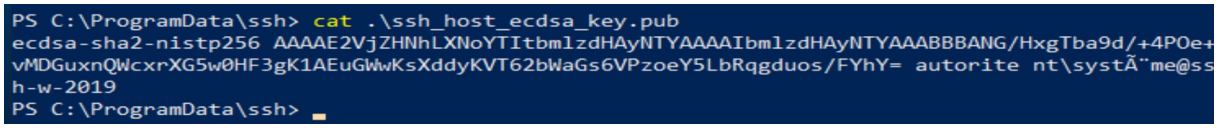




On peut vérifier comme auparavant l'envoie de la clé publique par le serveur ssh Windows au serveur linux sshclient
Sur sshclient

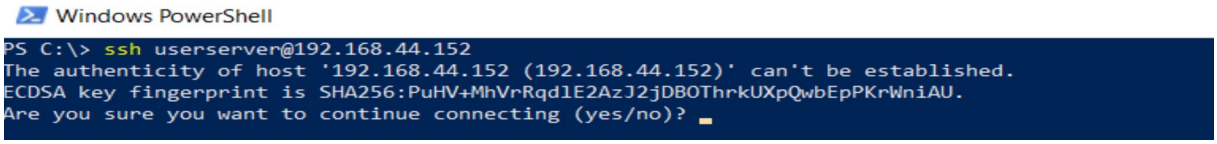


Sur w2019



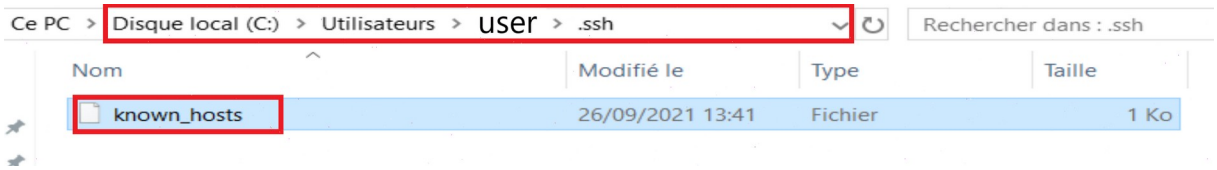
On remarque que c'est la même clé

On peut tenter d'établir une connexion de Windows 2019 vers un serveur linux
On ouvre une connexion ssh sur un terminal PowerShell ou dos



Yes pour confirmer puis on tombe sur le terminal du serveur ssh sur linux on peut vérifier aisément que le serveur ssh a envoyé sa clé publique



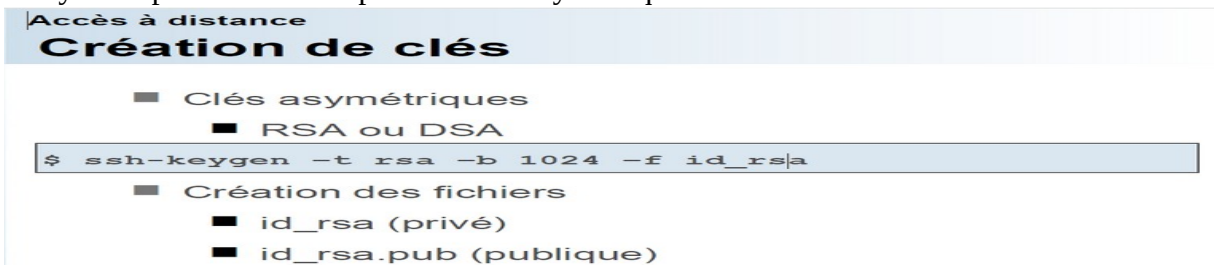


c- Votre machine physique Windows 10
Idem que Windows 2019

C- Création d'une identité NUMERIQUE pour les comptes utilisateurs ssh en créant une paire de clés asymétrique

1- Création d'une identité NUMERIQUE pour userclient en créant une paire de clés asymétrique sur sshclient

Création d'une paire de clés asymétrique la clé **privée** est protégé par un **mot de passe**
La syntaxe pour créer cette paire de clés asymetriques



s

#ssh-keygen -t algorithme -b la taille de la clé -f le chemin ou la clé sera stockée

#ssh-keygen -t rsa -b 1024 -f /home/userclient/uc_rsa

```
root💀sshclient:~# ssh-keygen -t rsa -b 1024 -f /home/userclient/uc_rsa
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/userclient/uc_rsa
Your public key has been saved in /home/userclient/uc_rsa.pub
The key fingerprint is:
SHA256:EZIqNtPh2rpv4Wg5ytNSaQb0zQFao4FFaLGFNhk0qAI root@sshclient
The key's randomart image is:
+---[RSA 1024]----+
|+XB=. ...        |
|EBO o... .       |
|*+oooo. .        |
|o.=.+o   ..      |
|...*.   S        |
|  .=o            |
|  == .           |
|.o*.o            |
|.++=.            |
+----[SHA256]-----+
```

Je vérifie la création de la paire de clé

```
root💀sshclient:~# ls /home/userclient/
uc_rsa  uc_rsa.pub
```

Je peux changer le mot de passe d'une clé privée

```
root💀sshserver1:~# ssh-keygen -p -f /home/user/rsa_user
```

- J'affiche la clé privée

```
userclient💀sshclient:~$ cat uc_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1Ni1jdHIAAAAGYmNyeXB0AAAAGAAAABDZelRmg2
KxpLKVuOER+WsEAAAAEAAAAAEAAACXAAAAB3NzaC1yc2EAAAADAQABAAAAgQDdKVWTmNOC
WpBq4oMLbEqsfAES+ZDvDhdBHzYXW1vUT+U/Xg5RgrjSJKFH3GZ3Ie9MHznwZv4uXzRGg9
hcrvzMtIdJbDpzpjk8Z6pPhW71W9u8NJqeRfidoh//7M6+v/f0XDxkrIJtT3yXnuGHulf8
OSYJRrXmNa3YP+J2SgFwQwAAAhDg3VDRhbrbrup/AA/Lnrljuhqjinpg0JxRPK4ZLo8u9k
NakQbtanU62cd7QicFxNnYPjofSyKCTZRCC/6vrxaQLOyuWw+YU0jd9qpZSF5Q+4D6dANj
or1VOQ0ptwxwEWA98YpfJOxWpgQsNPWx+tXyK1zgX40ZxRxrUuQLHX6oKSDXAvcjKi+YHf
HPuPVarH59QEYqBzep06HQxdhjtezNn1hFGsaJobT9BF2Lbxr+mIBY/Gdgb+TOyLPtzzcK
YASh7imzyc7xl9ufnVZ4h5FK9chxbrz2Q9mZb25CyF5ryJpqtb7cRDdnEA1ID8MLgqyAZV
0mrOhlD3LS9Dx7FJqiTY1ewqTpp3JvDGGVlwDzOPVPwVSG45hPNk01Fqp6rmmZeeRGg3ny
pNTk+fTXAaZNOZ8DcjIGKmZLz7zpCooPuZw8b39OZI44YuhH/1EehXuZGq80DzoEjY01ch
27IHRFshpBzeJIKs1+Y4zfvGAkKlnP/w9cmjicodXS6OsPx1277UTWT9tOCqNDo2G7wpv2
hjtZ7uIlEJ8N7Uw9ZtmBHM/v7NXX4pDXauxLzMMUlyW8+84suiNWpAKaNIjcX5kpz1t28t
S5yMZVlsaKrhka+2fG52T0dO/9GwmKtsjY3Q4HOT8/UCyeYdvkgqmphayy6+p3s3hrdbe5
ck9xBYCfPBCIXJkDhfnrFbCHCcPrhpq=
-----END OPENSSH PRIVATE KEY-----
```

- J'affiche la clé publique

```
userclient💀sshclient:~$ cat uc_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAAAgQDdKVWTmNOCWpBq4oMLbEqsfAES+ZDvDhdBHzYXW1vUT+U/Xg5RgrjSJKFH3GZ3Ie9
MHznwZv4uXzRGg9hcrvzMtIdJbDpzpjk8Z6pPhW71W9u8NJqeRfidoh//7M6+v/f0XDxkrIJtT3yXnuGHulf8OSYJRrXmNa3YP+J2Sg
FwQw== userclient@sshclient
userclient💀sshclient:~$
```

Je vérifie l'empreinte numérique de ma clé publique

```
root💀sshclient:~# ssh-keygen -l -E sha256 -f /home/userclient/uc_rsa.pub
1024 SHA256:EZIqNtPh2rpv4Wg5ytNSaQb0zQFao4FFaLGFNhk0qAI root@sshclient (RSA)
```

2- J'envoie la clé publique à userserver sur sshserver

```
userclient💀sshclient:~$ ssh-copy-id -i uc_rsa.pub userserver@172.20.2.30
```

```
userclient💀sshclient:~$ ssh-copy-id -i uc_rsa.pub userserver@192.168.44.152
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "uc_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
userserver@192.168.44.152's password:

Number of key(s) added: 1

Now try logging into the machine, with:   "ssh 'userserver@192.168.44.152'"
and check to make sure that only the key(s) you wanted were added.
```

On vérifie sur le serveur sshserver si la clé publique est envoyée elle doit être stoker dans le répertoire personnel de l'utilisateur userserver

```
userserver💀sshserver:~$ pwd
/home/userserver
userserver💀sshserver:~$ ls -a
.  ..  .bash_aliases  .bash_history  .bash_logout  .bashrc  .config  .profile  .ssh  .viminfo
userserver💀sshserver:~$ ls .ssh
authorized_keys
userserver💀sshserver:~$ cat .ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAAAgQDdKVWTmNOCWpBq4oMLbEqsfAES+ZDvDhdBHzYXW1vUT+U/Xg5RgrjSJKFH3GZ3Ie9
MHznwZv4uXzRGg9hcrvzMtIdJbDpzpjk8Z6pPhW71W9u8NJqeRfidoh//7M6+v/f0XDxkrIJtT3yXnuGHulf8OSYJRrXmNa3YP+J2Sg
FwQw== userclient@sshclient
userserver💀sshserver:~$
```

On remarque qu'elle correspond à celle affiché précédemment sur le serveur sshclient

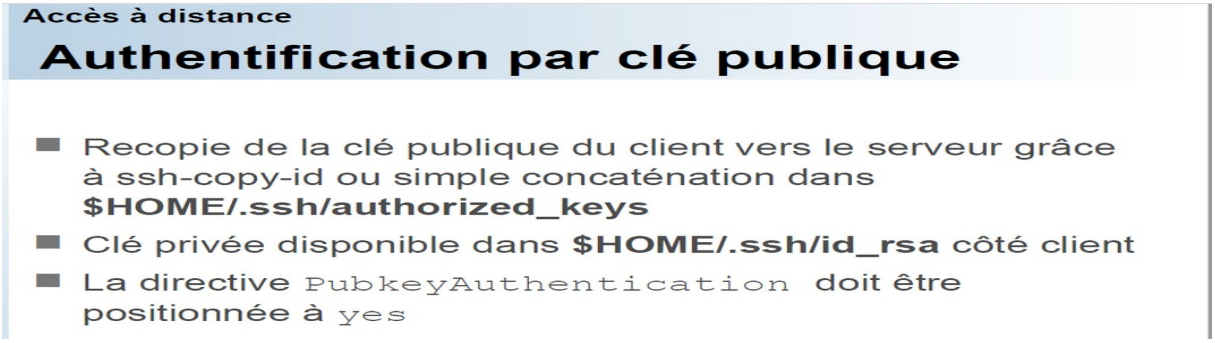
3- Je me connecte en ssh sur sshserver avec le compte sshserver ; la machine distante va remarquer qu'elle possède la clé publique attaché à mon compte elle va m'envoyer un message chiffré (challenge) je le déchiffre le message et je l'envoi je prouve que j'ai en ma possession la clé privée donc je certifie mon identité dans le cas contraire je ne pourrais pas me connecter

```
userclient💀sshclient:~$ ssh -i uc_rsa userserver@192.168.44.152
Enter passphrase for key 'uc_rsa': useruc
```

Remarque

Si on ne change pas le nom par défaut de la clé privé on n'est pas obligé de mettre le **-i nom de la clé privé** c'est-à-dire si dans l'exemple on change le nom de la clé privé de uc_rsa à id_rsa on peut juste taper :

```
#ssh user@172.20.2.30
```



D- Passphrase et agent SSH

Ssh-agent est un outil coté client ;**Ssh-agent** permet de **rajouter dans la mémoire** de mon agent ma clé privée tout en déverrouillant la clé privée pendant un certain temps
La commande *ssh-add* permet de charger la clé privée en mémoire.

```
root💀server1:~# man ssh-add
SSH-ADD(1)   BSD General Commands Manual   SSH-ADD(1)

NAME
     ssh-add – adds private key identities to the
     OpenSSH authentication agent

SYNOPSIS
     ssh-add [-cDdKkLlqvXx] [-E fingerprint_hash]
             [-S provider] [-t life] [file ...]
     ssh-add -s pkcs11
     ssh-add -e pkcs11
     ssh-add -T pubkey ...
```

l' ***agent*** est un programme qui garde les clés en mémoire afin de les déverrouiller *qu'une seule fois.*
Il y'a deux méthodes pour démarrer *ssh-agent* :

a- **eval** `ssh-agent`- cela exécute l'agent en arrière-plan et définit les variables d'environnement appropriées pour l'instance de shell *actuelle.*
b- **exec** ssh-agent bash- démarre une *nouvelle* instance du bashshell, en remplaçant l'actuelle.

La deuxième méthode est parfois préférée, car elle arrête automatiquement ssh-agent lorsque on arrête la session.
Après il faut utiliser ***ssh-add*** , pour déverrouiller les clés et les associés à l'agent,
les connexions *ssh, scp, sftp …* pourront tirer profit de cette méthode afin de simplifier les connexions.

a- Création d'une nouvelle instance bash

exec ssh-agent bash- démarre une *nouvelle* instance du bashshell, en remplaçant l'actuelle.

```
user💀sshclient:~$ exec ssh-agent bash
```

b- Charger la clé dans la mémoire de l'agent

Utiliser ***ssh-add***, déverrouille les clés (généralement ~/.ssh/id_*) et les charge dans l'agent, les rendant accessibles aux connexions *ssh* ou *sftp*.

```
userclient💀sshclient:~$ ssh-add uc_rsa
Enter passphrase for uc_rsa: useruc
Identity added: uc_rsa (userclient@sshclient)
userclient💀sshclient:~$
```

Pour vérifier on peut lister les clés en mémoire

```
userclient💀sshclient:~$ ssh-add -l
1024 SHA256:sJNv70kVxqI2+m1pZagLgQSJaeOtE2ZoAtujOMQOmFg userclient@sshclient (RSA)
```

c- Connexion ssh en utilisant l'agent ssh avec la Passphrase en mémoire

Je me connecte en ssh on me demande plus la clé passphrase

SSH

```
userclient💀sshclient:~$ ssh -i uc_rsa userserver@192.168.44.152
```

d- Autres options

Si je veux effacer des clés en mémoire l'agent ssh

```
userclient💀sshclient:~$ ssh-add -d uc_rsa
Identity removed: uc_rsa RSA (userclient@sshclient)
userclient💀sshclient:~$ ssh-add -l
The agent has no identities.
userclient💀sshclient:~$
```

Mémoriser la clé pour une durée limité

```
userclient💀sshclient:~$ ssh-add -t 60 uc_rsa
Enter passphrase for uc_rsa:
Identity added: uc_rsa (userclient@sshclient)
Lifetime set to 60 seconds
userclient💀sshclient:~$
```

Après 60 secondes la clé est effacée de la mémoire de l'agent

```
userclient💀sshclient:~$ ssh-add -l
The agent has no identities.
```

E- Créations du fichier config

Dans le cas où connait une multitude de serveurs le fichier config qu'on peut créer dans le dossier ./ssh nous facilitera la tache car au lieu de taper la commande

#ssh user@adresse ip du serveur

On peut juste entrer le nom de la machine

#neptune par exemple c'est plus facile car mémoriser toute les adresses ip est fastidieux

a- Créer le fichier config dans .ssh

```
userclient💀sshclient:~$ cd .ssh
userclient💀sshclient:~/.ssh$ touch config
userclient💀sshclient:~/.ssh$ ls
config  known_hosts
```

b- Modifier le fichier config

```
userclient💀sshclient:~/.ssh$ vim config
```

```
host sshserver
        user userserver
        port 22
        identityfile /home/userserver/uc_rsa
~
```

c- modifier le fichier hosts

```
root💀sshclient:~# vim /etc/hosts
```

```
127.0.0.1       localhost
127.0.1.1       sshsclient
192.168.44.152  sshserver

# The following lines are desirable for IPv6 capable hosts
::1     localhost ip6-localhost ip6-loopback
ff02::1 ip6-allnodes
ff02::2 ip6-allrouters
```

Maintenant on peut se connecter sur notre serveur

```
userclient💀sshclient:/root$ ssh sshserver
```

F- La double Authentification google

On commence par installer le paquet **libpam-google-authenticator** sur le serveur ssh

```
root💀sshserver:~# apt install libpam-google-authenticator
```

On modifie le fichier sshd dans le dossier /etc/pam.d/

```
root💀sshserver:~# vim /etc/pam.d/sshd
```

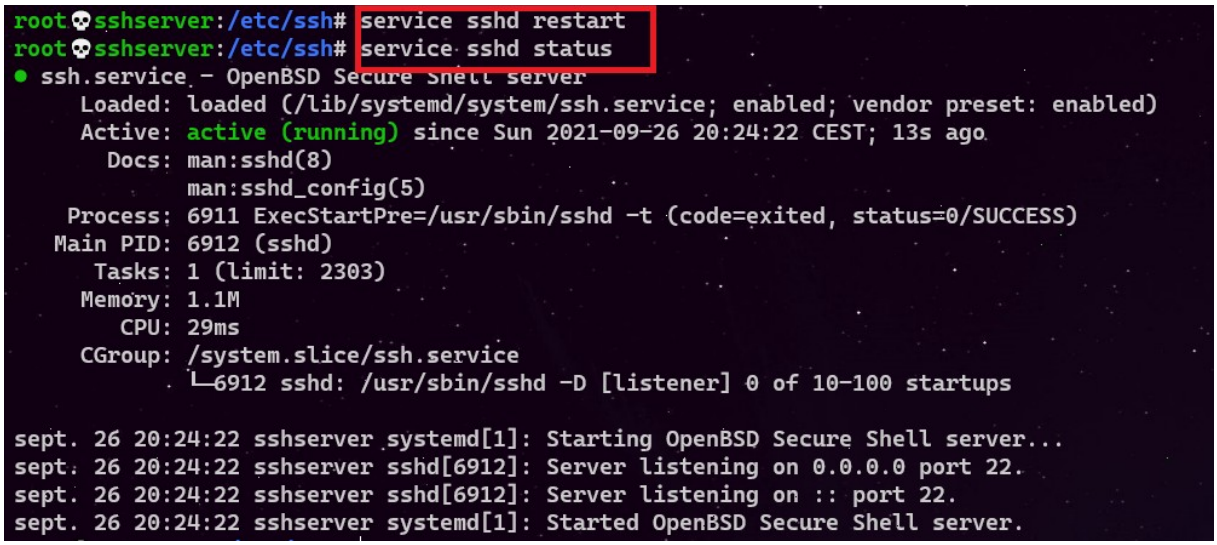
On rajoute à la fin dans ce fichier le bloc suivant

```
#Authentification Google
auth required pam_google_authenticator.so
```

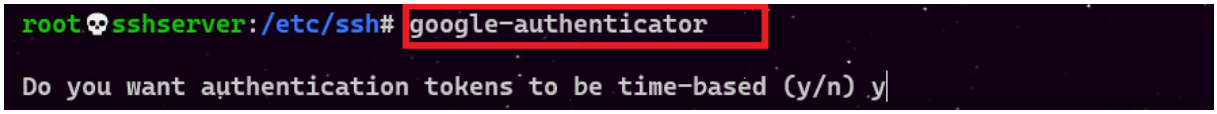
On modifie aussi le fichier sshd_config dans /etc/ssh/

```
root💀sshserver:/etc/ssh# vim sshd_config
# Change to yes to enable challenge-response passwords (beware issues with
# some PAM modules and threads)
ChallengeResponseAuthentication yes
```

On redémarre le service sshd puisque on modifier le fichier sshd_config
En même temps on vérifie si le service a bien redémarrer



On lance google-authenticator



Attention il faut lancer google-authenticator pour chaque utilisateur si on veut appliquer la double authentification à tous les utilisateurs



Sur le smartphone il faut télécharger sur le store l'application  Authenticator

Synchronisation automatique

Une horloge possède une dérive inhérente dû au quartz de la carte mère, l'horloge a tendance à se décaler de l'heure officielle (plusieurs secondes par jour pour certaines cartes mère). Pour éviter ce décalage, on utilise le protocole NTP pour synchroniser notre serveur avec un serveur de temps.

Il faut d'abord installer les paquets ntp et ntpdate avec la commande suivante

```
root💀server1:~# apt install ntp ntpdate
```

Il faut configurer NTP on peut utiliser :

Serveurs NTP du projet **pool.ntp.org**

Pour la France, les serveurs NTP a utiliser sont : **server 0.fr.pool.ntp.org server 1.fr.pool.ntp.org server 2.fr.pool.ntp.org**

Donc on va modifier le fichier /etc/ntp.conf et de remplacer les lignes correspondantes aux adresses des serveurs NTP par celles ci-dessus.

On peut régler l'heure manuellement avec la commande suivante :

```
root💀server1:~# ntpdate pool.ntp.org
 8 Oct 18:45:44 ntpdate[1920]: adjust time server 45.33.84.208 offset -0.021789 sec
```

On vérifie si le service ntp est démarrer

```
root💀server1:~# service ntp status
● ntp.service - Network Time Service
     Loaded: loaded (/lib/systemd/system/ntp.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2021-10-08 18:49:03 CEST; 27s ago
       Docs: man:ntpd(8)
    Process: 1930 ExecStart=/usr/lib/ntp/ntp-systemd-wrapper (code=exited, status=0/SUCCESS)
   Main PID: 1936 (ntpd)
      Tasks: 2 (limit: 2303)
     Memory: 948.0K
        CPU: 41ms
     CGroup: /system.slice/ntp.service
             └─1936 /usr/sbin/ntpd -p /var/run/ntpd.pid -g -u 106:112

oct. 08 18:49:05 server1 ntpd[1936]: Soliciting pool server 95.81.173.155
oct. 08 18:49:05 server1 ntpd[1936]: Soliciting pool server 37.187.105.73
oct. 08 18:49:06 server1 ntpd[1936]: Soliciting pool server 129.250.35.251
oct. 08 18:49:06 server1 ntpd[1936]: Soliciting pool server 162.159.200.1
oct. 08 18:49:07 server1 ntpd[1936]: Soliciting pool server 212.83.179.156
oct. 08 18:49:07 server1 ntpd[1936]: Soliciting pool server 162.159.200.123
oct. 08 18:49:07 server1 ntpd[1936]: Soliciting pool server 80.74.64.2
oct. 08 18:49:08 server1 ntpd[1936]: Soliciting pool server 51.255.197.148
oct. 08 18:49:09 server1 ntpd[1936]: Soliciting pool server 78.196.167.192
oct. 08 18:49:10 server1 ntpd[1936]: Soliciting pool server 92.243.6.5
root💀server1:~#
```

G- Création d’une bannière

Le but est d’avoir une bannière d’accueil au cours de notre connexion Ssh
Pour réaliser cette bannière il faut :

a- Créer un fichier qui va présenter cette bannière on peut lui donner le nom qu’on veut dans notre cas je lui ai donné le nom de banner

```
root💀server1:~# vim banner
```



b- Il faut renseigner le chemin du fichier banner dans sshd_config

```
root💀server1:/etc/ssh# vim sshd_config
```

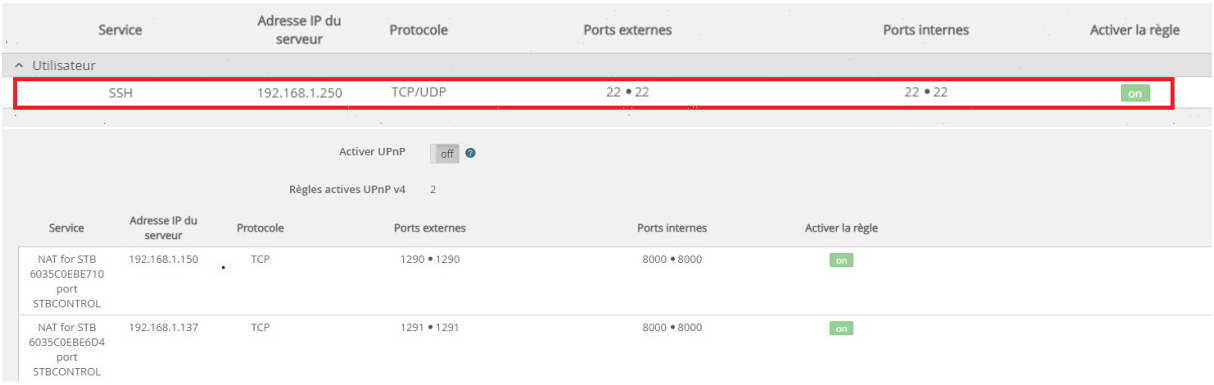
```
# no default banner path
Banner /root/banner
```

H- Connexion ssh avec une solution mobile

a- Connexion avec une redirection de port

Maintenant on va essayer d’accéder à notre serveur ssh à partir de l’extérieur en utilisant notre adresse publique.
Tout d’abord :
Attention mettez votre machine en bridge au lieu du Nat

1- Il faut accéder à la boxe internet et ouvrir le port 22 en créant une redirection de port



2- Ensuite il faut déterminer notre adresse publique soit à partir de la boxe ou un site internet http://www.whatismyip.com

3- Après sur notre smartphone on télécharge un client ssh sur Google Play google store **Juicessh**

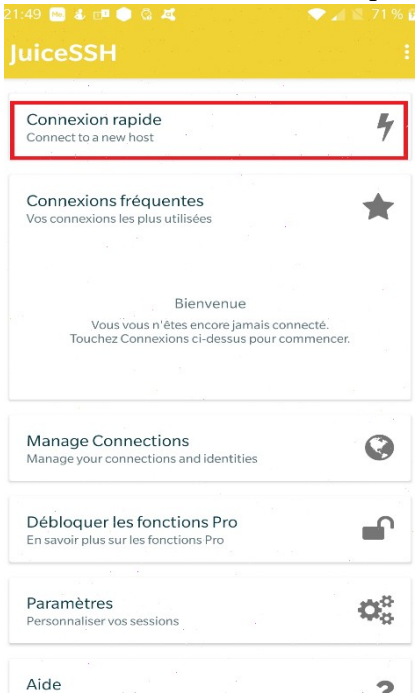


JuiceSSH - SSH Client

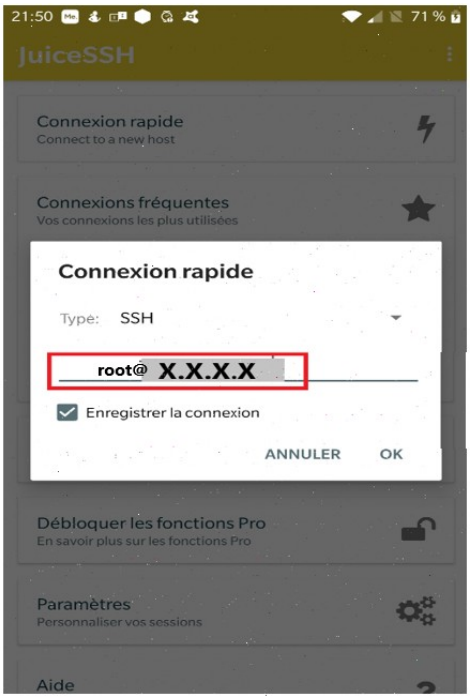
4- Sur notre smartphone il faut qu'on se mette en 4G et non en wifi.
On peut effectuer cette procédure en wifi sur notre smartphone et non en 4G.

5- On ouvre l'application et on commence à établir notre connexion ssh

On sélectionne connexion rapide

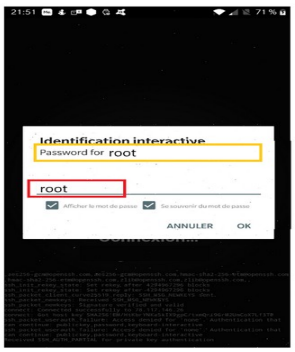


ssh root@ adresse publique

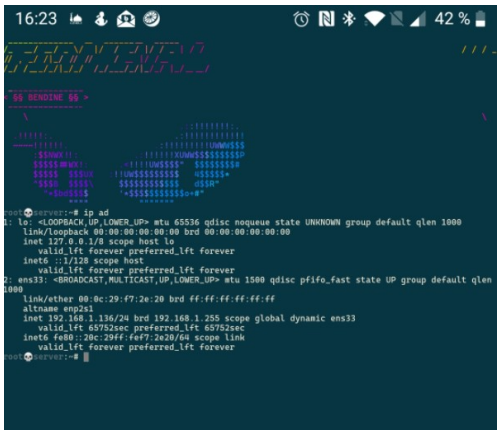


Le serveur nous envoie l'empreinte de sa clé publique
On constate que c'est la même que celle qu'on a calculé
Sur le serveur
EM/HtKbrYNKa5kIX9gpC/txmQri9GrM2UmCoX7Lf3TE rentre le mot de passe root



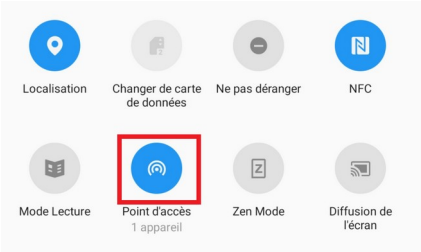


Après on tombe sur notre console VMWare



b- Connexion avec un partage de connexion 4G

- Faite un partage de connexion à partir de votre smartphone



- Connectez-vous en wifi sur votre machine physique en utilisant le wifi correspondant au partage de connexion de votre smartphone
- Mettez votre serveur SSH en Bridge pour avoir une adresse IP du même réseau que la machine physique et le smartphone
- Installez un terminal sur le smartphone :

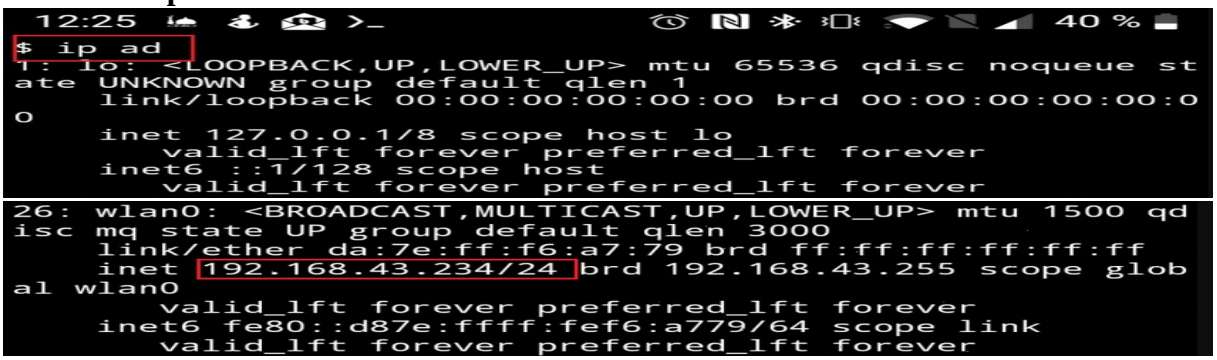
Exemple de terminal :

Termux
Fredrik Fornwall

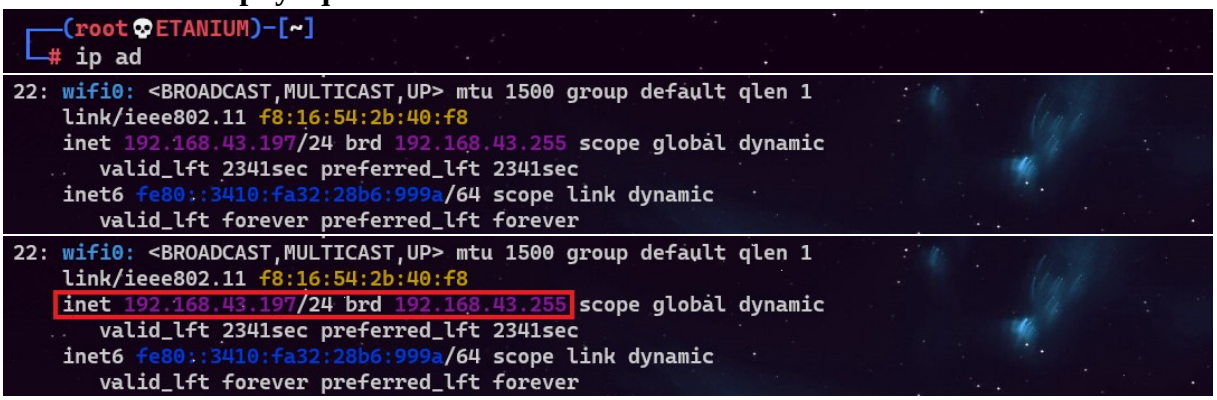
- Relevez l’adresse IP smartphone, machine physique et la VM serveur SSH

Ces adresses (réseau 192.168.43.0/24) sont celles du partage de connexion 4G du smartphone : ce test se fait en dehors du plan d’adressage Sitka.protocole de communicationclés de chiffrementsegmentsTCP

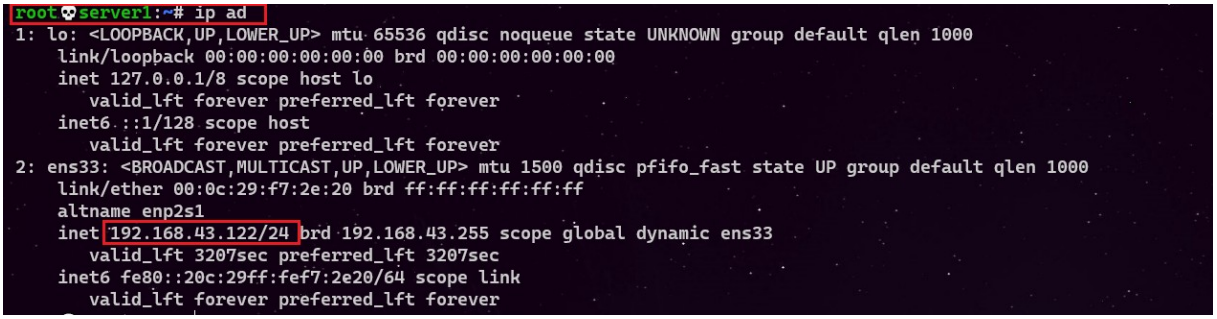
IP du smartphone : 192.168.43.234 /24



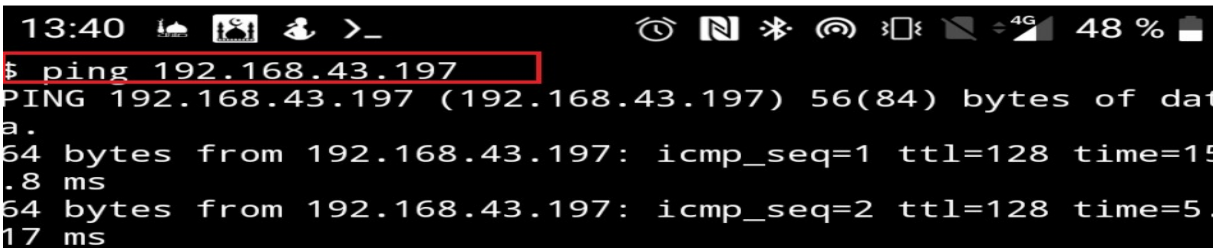
IP de la machine physique : 192.168.43.197 /24



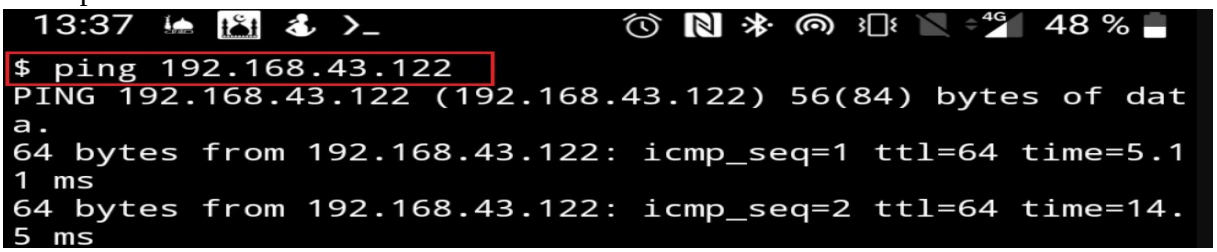
IP de la machine VM serveur SSH : 192.168.43.122 /24



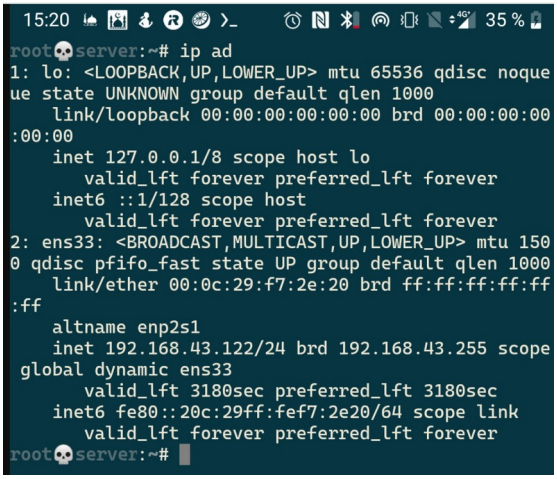
- Faite un test de ping entre smartphone, machine physique et la VM serveur SSH
Smartphone à Machine physique



Smartphone àServeur SSH



- Utilisez maintenant votre application client SSH sur votre smartphone pour se connecter la VM serveur SSH, avec **ip ad** on vérifie bien qu'on est sur le serveur ssh

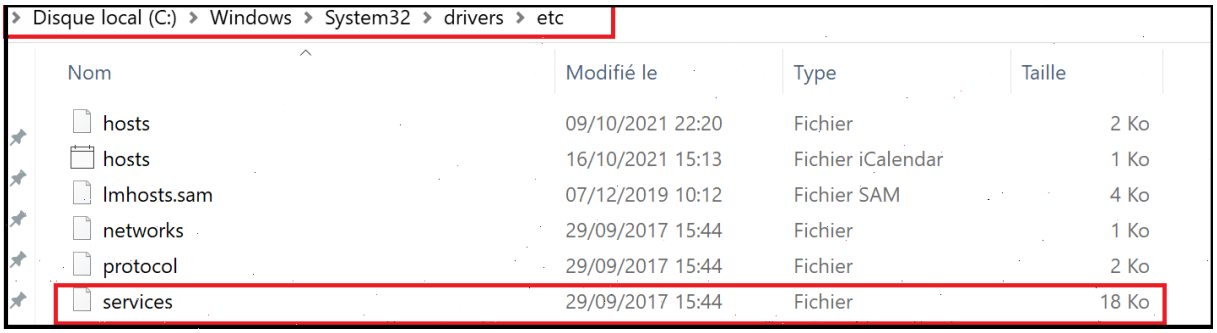


I- Les Ports de communication:

Un numéro de port est codé sur 16 bits, ce qui fait qu'il existe un maximum de 2^16,soit **65 536** ports distincts par machine. Ces ports sont classés en 3 catégories en fonction de leur numéro:

- les numéros de port de 0 à 1 023 correspondent aux ports "bien-connus" (**well-known ports**), utilisés pour les services réseaux les plus courants.

- sous Windows, ce fichier est dans **C:\Windows\System32\drivers\etc**



- Sous linux Le fichier services indiquant la liste des services dits well-known, est dans /etc ;

```
root💀server1:/etc# ls services
services
```

- les numéros de ports de 1 024 à 49 151 correspondent aux ports enregistrés (registered ports), assignés par l'IANA
- les numéros de ports de 49 152 à 65 535 correspondent aux ports dynamiques, utilisables pour tout type de requêtes TCP ou UDP autres que celle citées précédemment.

Lorsqu'un logiciel client veut dialoguer avec un logiciel serveur, aussi appelé service, il a besoin de connaître le port écouté par ce dernier. Les ports utilisés par les services devant être connus par les clients, les principaux types de services utilisent des ports qui sont dits réservés. Par convention, ce sont tous ceux compris entre 0 et 1 0231 inclus et leur utilisation par un logiciel serveur nécessite souvent que celui-ci s'exécute avec des droits d'accès particuliers. Les services utilisant ces ports sont appelés les services bien connus ("Well-Known Services").

Le fichier services indique la liste de ces services dits well-known. Sous UNIX, ce fichier est directement dans /etc ; sous Windows, ce fichier est par défaut dans C:\Windows\System32\ drivers\etc. Les services les plus utilisés sont :

Toutefois, ces conventions ci-dessus peuvent ne pas être respectées pourvu que le client et le serveur soient cohérents entre eux et que le nouveau numéro choisi ne soit pas déjà utilisé
Pour afficher les port on utilise netstat

```
#netstat -paunt
```

- -a : Tous les ports
- -t : Tous les ports TCP
- -u : Tous les ports UDP
- -l : Tous les ports en écoute
- -n : Affiche directement les IP. Pas de résolution de nom.
- -p : Affiche le nom du programme et le PID associé.